

Demonstration of Selective Forwarding Attack and Detection Mechanism in UnetStack3

B. R. Chandavarkar

*Wireless Information Networking Group (WiNG)
Department of Computer Science and Engineering
National Institute of Technology Karnataka
Surathkal, Mangalore, India
brcnitk@gmail.com*

Nakul Suresh

*Department of Computer Science and Engineering
National Institute of Technology Karnataka
Surathkal, Mangalore, India
nakulsuresh98@gmail.com*

Abstract—Underwater networks provide new possibilities that can greatly improve our ability to observe and predict the ocean. Underwater acoustic sensor networks (UASNs) are frequently used in uncharted and unexplored waters, making them vulnerable to various security threats. Applications based on UASNs such as coastal defense, pollution monitoring, assisted navigation to name a few, require secure communication. Various communication protocols and algorithms have been developed to enable collaborative monitoring tasks. However, These protocols often place a low priority on security. Impersonating nodes or illegally manipulating routing information can affect the entire network, making UASNs vulnerable to attacks such as selective forwarding, sinkhole attack, Sybil attack, acknowledgement spoofing and HELLO flood attack. The lack of security against such threats is startling, since security is indeed an important requirement in many emerging civilian and military applications.

This paper takes a look at the selective forwarding attack, examines a model as well as a detection mechanism created in UnetStack3.

Index Terms—Underwater acoustic sensor network(UASN), Security, Selective Forwarding(SF), UnetStack3, Wireless Sensor Network(WSN)

I. INTRODUCTION

A. Basics of Underwater Networks

Underwater networks are now emerging as a hot topic among researchers due to a surge of novel applications in scientific exploration such as oceanography, earthquake and tsunami detection, as well as commercial exploitation like homeland security and industrial uses. Underwater networks use acoustics for communication and are called Underwater Acoustic Sensor Networks(UASNs) for this reason. Each transmitting/receiving component is referred to as a Node.

UASNs plays a major role when it comes to applications such as ocean exploration, earthquake and tsunami detection, homeland security and meteorological studies. However, the technology involved in this is not as advanced as its terrestrial counterparts since designing and deploying UASN comes with their own challenges, some of which are given in [1]. For example, obstacles like ships,

boats and marine life can tamper with the nodes used. Water as a communication channel is very unreliable. Most nodes are battery-operated due to sparse deployment, and computationally intensive algorithms tend to drain power quickly. Communication speed is slow due to vast distances involved.

Attackers can use these issues to their advantage and hijack UASNs. A malicious node can drop packets at will without causing suspicion, or a legitimate node might be flagged as a suspicious node due to an unreliable channel. Many protection algorithms commonly used in terrestrial networks are too power-intensive to directly be implemented in UASNs. Detecting malicious network activity is further complicated by delayed transmission and reception times.

As a result, security is a major issue faced by UASNs currently. Fast and efficient algorithms are the need of the hour. This paper looks into the behavior and possible mitigation strategies for the selective forwarding attack.

B. Selective Forwarding

Most multi-node UASNs involve some form of routing as mesh networks are quite expensive to deploy and maintain. The role of a routing node is to receive packets from the sender and transmit it to the recipient. However, a malicious routing node might drop some packets that are to be sent, or in extreme cases not transmit any of the received packets. Such an attack is referred to as Selective Forwarding, where a malicious routing node drops any packets received.

Such malicious nodes drop packets either selectively or randomly. This increases the packet loss rate of the network and adversely impacts the throughput rate as well as the reliability rate of said network. There are 2 ways by which a malicious node can attack the network, as explained in [2].

- 1) Insider Attack
- 2) Outsider Attack

In an Outsider Attack, a malicious node can either spoof the ID of a legitimate node or can jam the route between

two nodes. It may then force these nodes to send packets through it, which the attacker can selectively forward.

In an Insider attack, an attacker might be granted access to the network by posing as a legitimate node. Once it acquires any sensitive information such as routing information or keys, it can launch an attack by unlawfully manipulating the routing table.

C. Types of Selective Forwarding Attacks

While the underlying methodology is the same, selective forwarding attacks differ in terms of objectives and what they selectively forward. A few such types are given below, as presented in [2]

- A malicious node can prevent communications between the base station and any authorized nodes present in the network. Such an attack is called a **Denial of Service** attack. If a node with a large number of connections gets infected, a **Black Hole** attack can occur, where almost all packets sent to this node get dropped.
- A malicious node, upon receiving a packet may choose to drop the received packet and send its own packet instead. Such an attack is called a **Neglect and Greed** attack. Neglect and Greed attacks can be highly dangerous in trust-based networks, since the overall reliability of the network is affected, making the attacker hard to detect.
- A malicious node may sometimes choose to delay the transmission of packets between nodes. While this on its own is relatively harmless, it can be used in conjunction with a sinkhole attack to inflict devastating damage on a network.
- A malicious node receives a packet to be forwarded, it may drop the received packet and send a false acknowledgement to the transmitting node. Such an attack is called a **Blind Letter** attack.

There are a variety of solutions available for the above-mentioned attacks, but often these require high amounts of energy and bandwidth, which is a luxury that cannot be afforded in the case of UASNs. Hence, the solution approach for this attack has been chosen taking this and feasibility of implementation into account.

The rest of the paper is organised as follows: Section II presents a review on various related works of literature. Section III presents the design of attack model and detection approach. Section IV explains the implementation on both in UnetStack3. Section V presents results of a performance evaluation done on both implementations. Section VI concludes the paper and provides suggestions for future work.

II. RELATED WORK

The authors of [3] elaborate on the various types of attacks prevalent in underwater networks along with the network stack they affect. The paper authored by [4] talks about the vulnerabilities present in underwater networks

along with brief descriptions of various attacks. These papers however only presented general purpose solutions for the same, such as improved cryptographic measures, better authentication, greater reliability and so on without giving any further details about the same.

The paper by [5] contains an extensive literature survey of the various trust-based defense mechanisms available. It begins by talking about how trust is defined, various characteristics of trust such as reflexivity, subjectivity etc. It then describes a few commonly used mathematical trust models, followed by a brief list of the challenges faced by trust schemes in WSNs.

Young and Cho in [6] talk about the Beta Trust model and how it works in defending against selective-forwarding(SF) based denial-of-service(DoS) attacks. It then explains via counterexample on how an attacker can bypass this defense mechanism to launch an SF based DoS attack with relevant mathematical analysis.

Some possible drawbacks noticed were the potential to consume a lot of energy and bandwidth as the algorithm must run continuously. Another possible drawback is that a potential attacker can lower its attack rate such that it doesn't cross the threshold for breach-of-trust. This threshold is set relatively high as doing so resulted in a significant reduction in misidentifying attackers. Hence, there is a trade-off involved here. Moreover, many of these were centralized models, which are not always feasible to implement in UASNs.

The group who authored [7] did another literature survey on 16 different mitigation approaches for selective forwarding attacks. They compared on aspects like efficiency, reliability, energy consumption, distributed/centralized, accuracy etc. Of these, the method by [8] was the simplest to implement in the given time period. It was distributed, had 90 percent accuracy, and had a reasonable efficiency and energy consumption.

When it comes to implementing a mitigation approach, encoding, decoding and transmitting these payloads can consume a lot of energy. Hence, our approach only takes care of detection. The papers written by [9] and [10] gave insights on how to build and simulate networks on UnetStack3.

As mentioned above, there are several mitigation strategies for SF. These are tested on OMNeT, Castalia or other such simulators. These simulators are designed to simulate WSNs. This paper is the first ever to demonstrate the attack and provide a mitigation approach on UnetStack3. UnetStack3 is a simulation software designed specifically for UASNs. Its agent-based architecture allows easy development and testing of various protocols. It also allows simulation on real underwater nodes, which can be controlled remotely from a desktop or laptop [11].

There is a dearth of papers covering simulations in UnetStack3, especially in the domain of security. This paper seeks to address that by providing one such implementation. It can be deployed on live nodes without

much modification, which should simplify its adoption for both commercial and non-commercial uses.

III. DESIGN OF SELECTIVE FORWARDING AND ITS DETECTION

A. Attack Model

An expanded version of a simple 4 node network created on UnetStack3 is used to demonstrate the attack.

The network consists of a source node, a destination node, and two intermediate nodes. One of the intermediate nodes is designated as malicious. The network topology is given in Figure 1.

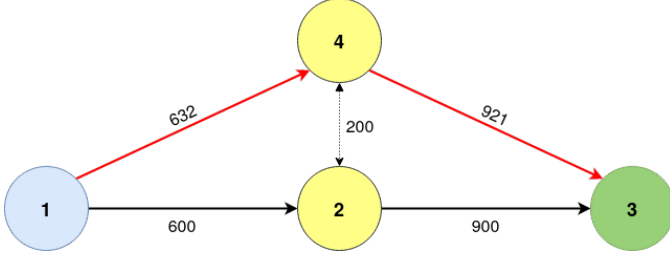


Fig. 1. Network topology to simulate selective forwarding

In the figure, node 1 is the source node, coloured blue. Node 3 is the destination node coloured in green, and nodes 2 and 4 are the intermediate nodes coloured in yellow. Node 4 is the designated malicious node in this model. The links in black show the legitimate routes in the network. The links in red show the routes made by the malicious node. The number on each arrow indicates the distance between two nodes in metres. The double headed arrow is simply used to show the distance between 2 and 4. No connection exists between them.

Initially, node 4 will spoof the ID of node 2, which would allow it to receive packets. Then, node 4 will alternately drop and forward the packet it receives while node 2 will be left out and made dormant. A flowchart showing the course of the program is given in Figure 2.

All nodes run the same UnetStack3 agent, which will be explained in section III.

B. Detection Approach

The detection approach chosen is an adaptation of the method proposed by [8]. It is a distributed algorithm that can employ any node in the network to be a monitor node. The following routing rules are followed by each monitor node:

Rule 1: *The monitor node waits to see if the destination node forwards the packet on the path to the sink. If not, it raises an alert packet with malicious factor α to the sender/source node.*

Rule 2: *The monitor node waits and detects the packet which has been forwarded on the path to the sink. It checks its two-hop neighbor knowledge to see if the destination node of the forwarded packet is on the right path to the sink. If*

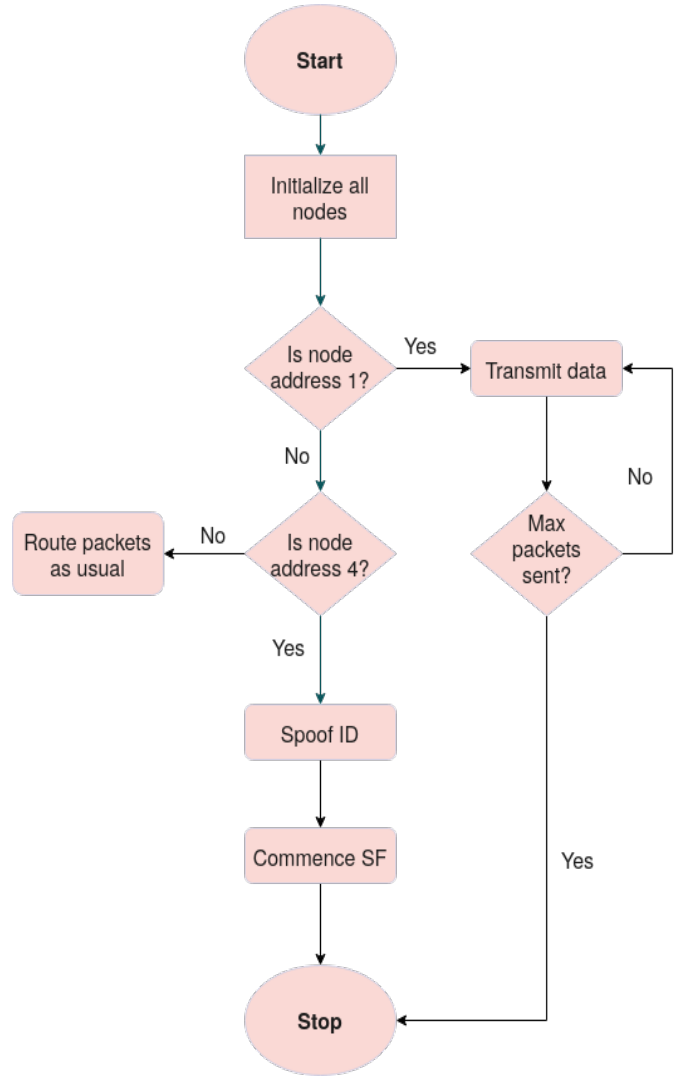


Fig. 2. Sequence of the attack model

not, it raises an alert packet with malicious factor β to the sender/ source node.

Given in Figure 3 is the algorithm as proposed by [8].

Each sensor node keeps scanning for packets for a specified timeout period. For each packet that a sensor node receives, it checks the packet's header. If the source and destination addresses are present in its neighbour routes, the node does nothing. Else it increases the malicious counter by α . If timeout occurs, it increases the counter by β .

Taking into account the capabilities of UnetStack3, the following modifications were made:

- The destination node functions as the monitor node.
- The rate of packet delivery is configured to limit packet collisions.
- Incrementing of α and β are done at the monitor itself.

Given in Figure 4 is a flowchart depicting the sequence of the detection strategy.

Monitor nodes

```

Repeat
<Listen packets in its radio range>
Check <packet header>
If {source & destination node's ID} in
1-hop neighbor list
    Buffer the packet with timer  $\tau$ 
    If {forwarded packet destination node
    ID in 2-hop neighbor list of buffered
    packet's source node ID}
        Delete buffered packet
    Else send an alert packet with  $\beta$  factor
And If finish buffer time  $\tau$ 
    Send an alert packet with  $\alpha$  factor
    & Delete buffered packet
Until No transmission

```

Fig. 3. Algorithm as proposed by [8]

IV. IMPLEMENTATION OF SELECTIVE FORWARDING AND DETECTION IN UNETSTACK3

A. Attack Model

The agent used has three main functions: **startup()**, **spoofId()** and **selectiveForward()**, along with a utility function **addroute(to, via)**. The **addroute()** function essentially creates a route from the node where its called to the destination node(to) with via as the intermediate node. T

1) *startup()*: The **startup()** function of a UnetStack3 agent is akin to that of the **main()** function in any regular computer program. Initially, the agent has to set up all the services and dependencies it will require to function normally. Once that is done, its working code will execute. [11]

Our agent first checks the address of the node it is called upon. If it happens to be the source node, the agent will set up routes to both destination nodes and then periodically transmit a message to them. If it happens to be the malicious node, it will activate its malicious code after a stipulated time period.

```

if(source node){
    addroute(3,2)
    (repeat every 9 seconds, {
        transmit(to:3, data:'hello there!')
    })
}

```

```

if(malicious node){
    spoof_id()
    selectiveForward()
}

```

2) *spoofId()*: The **spoofId()** function simply changes the address of the node to that of a legitimate node [11]

```
node.address = 2
```

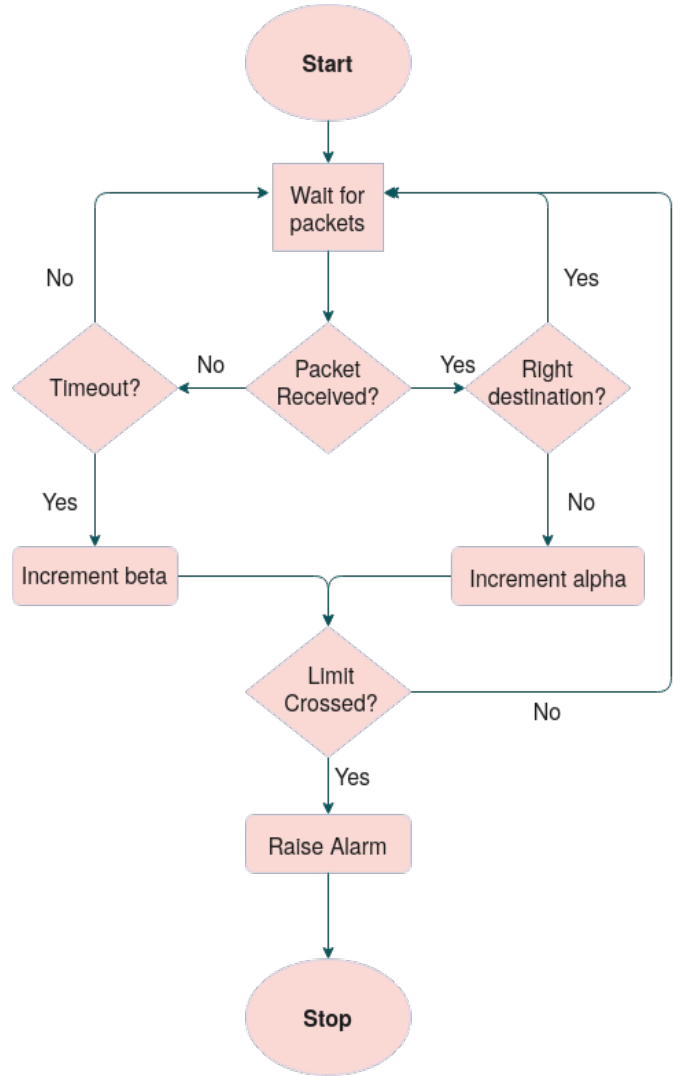


Fig. 4. Sequence of the detection strategy

3) *selectiveForward()*: **selectiveForward()** waits for a stipulated period (called the timeout period) to receive a packet from the source node. Once received, an internal flag variable decides whether to drop or forward the packet. It performs this by setting the node's **canForward** attribute to false and true respectively

```

def selectiveForward(){
    while(packet is received within timeout){
        flag = 0 or 1
        if(flag == 1){           // send packet
            node.canForward = true
        }else{                  // drop packet
            node.canForward = false
        }
    }
}

```

B. Detection Approach

The agent designed here has 2 primary functions: **detection()** and **alert()**, along with **startup()**. There is also a utility function **wakeup()** which makes the nodes sleep at random intervals to prevent packet collisions.

1) *detection()*: The *detection()* function waits for a specified timeout interval to receive packets. If it doesn't receive a packet, it increments β . If it does, it decrements β .

On receiving a packet, the node checks if the destination is correct. If yes, it decrements α and increments it otherwise. The incrementing and decrementing of α and β are done by the *alert()* function. The following are the parameters given to *alert()*:

- **BETA_INC** - Increase β .
- **ALPHA_INC** - Increase α .
- **BETA_DEC** - Decrease β .
- **ALPHA_DEC** - Decrease α .

Given below is the gist of the UnetStack3 code for the mechanism.

```
rxNtf = receive({data packet}, TIMEOUT)
if(Packet not received){
    alert(ALPHA_DEC, BETA_INC)
}else{
    if(Wrong destination){
        alert(ALPHA_INC, BETA_DEC)
    }else{
        alert(ALPHA_DEC, BETA_DEC)
    }
}
```

This code repeats continuously till there is no more transmission.

2) *alert()*: The alert function is called by the monitor node each time it seeks to change the values of either α or β . The alert function has two parameters *a* and *b*. *alert()* will change α and β based on the values given by *a* and *b*. The function ensures that α or β can never go below the minimum threshold value. When either α or β crosses the threshold value, it will raise an alarm, indicating the presence of a selective forwarding attack.

```
alpha += a
if(alpha < ALPHA_MIN)
    alpha = ALPHA_MIN

beta += b
if(beta < BETA_MIN)
    beta = BETA_MIN

if(alpha > ALPHA_THRESHOLD
or beta > BETA_THRESHOLD){
    Raise alarm
}
```

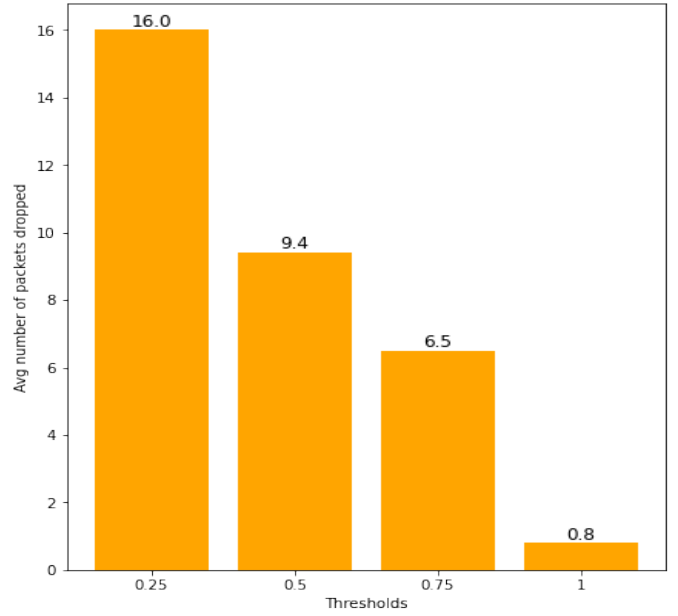


Fig. 5. Drop rate for different thresholds

V. RESULTS

A. Attack Model

The first subsection examines the performance of the attack model. The rate of SF can be controlled by setting the threshold for packet dropping. A pseudo-random number generator gives a value between 0 and 1. If it is above the threshold the packet is dropped, else it is forwarded. Figure 5 shows a graph showing number of packets dropped for different thresholds. Each value is the average of 5 runs. A total of 20 packets were sent in each run.

It can be seen that the higher the threshold value, the lesser the number of packets dropped. Furthermore, the rate appears to decrease smoothly, exhibiting a linear trend.

B. Detection Approach

Coming to the detection approach, due to the random nature of packet drops, there was no way to calculate the time taken to detect an attack in a deterministic manner. Moreover, the parameters for both can be configured to produce varying results. Hence, the quickest time taken to detect an attack for each threshold value is what is displayed here. This is measured by the total number of packets sent by the sender. Figure 6 shows the graph for the same.

α and β decrements have been set to zero so that convergence is guaranteed. The number of packets sent roughly doubles from 0.25 to 0.5, reducing as the threshold approaches 1. However the rate of change does not approach zero.

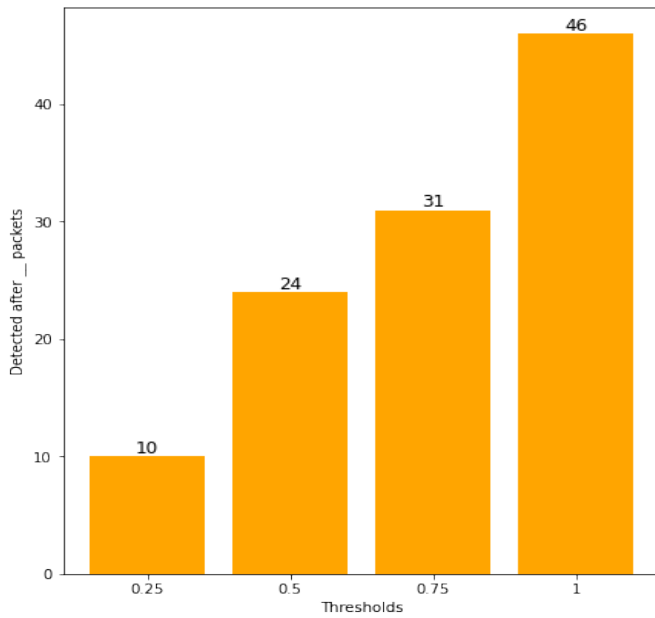


Fig. 6. Detection time for different thresholds

VI. CONCLUSIONS AND FUTURE WORK

We have demonstrated an attack model as well as a detection strategy for the selective forwarding attack implemented in UnetStack3. A performance evaluation of both has been done and the results shown in graphs.

The limitations of this paper are as follows. The alarm generates simply tells whether an attack is occurring or not. It does not tell about the number of nodes compromised, source of the attack or other useful information. The network topology implemented is a simple one, with a specific positioning of the attacker and monitor nodes.

Future implementations can work on making the detection approach more generalized. Work can be done to make the approach determine the source of the attack and other related information. Instead of just the receiver, the program can be generalized to run on multiple nodes to form a kind of neighbour monitoring system. Finally, the above approach is only a detection scheme. Work can

be done to implement a neutralization approach that will run on UnetStack3.

ACKNOWLEDGMENT

The authors thank the Science and Engineering Research Board (SERB), Govt. of India, for providing financial support (ref. no.EEQ/2018/001036).

REFERENCES

- [1] N. Li, J.-F. Martínez, J. M. Meneses Chaus, M. Eckert, A survey on underwater acoustic sensor network routing protocols, *Sensors* 16 (3) (2016). doi:10.3390/s16030414. URL <https://www.mdpi.com/1424-8220/16/3/414>
- [2] K. Mittal, *Selective forwarding Attack in wireless Sensor Network*, <https://www.geeksforgeeks.org/> [Accessed: 12th Nov] (2020).
- [3] G. Ateniese, A. Caposelle, P. Gjanci, C. Petrioli, D. Spaccini, Secfun: Security framework for underwater acoustic sensor networks, in: *OCEANS 2015 - Genova, 2015*, pp. 1–9. doi:10.1109/OCEANS-Genova.2015.7271735.
- [4] M. Domingo, Securing underwater wireless communication networks, *Wireless Communications, IEEE* 18 (2011) 22 – 28. doi:10.1109/MWC.2011.5714022.
- [5] Y. Yu, K. Li, W. Zhou, P. Li, Trust mechanisms in wireless sensor networks: Attack analysis and countermeasures, *Journal of Network and Computer Applications* 35 (2012) 867–880. doi:10.1016/j.jnca.2011.03.005.
- [6] Y. Cho, G. Qu, Detection and prevention of selective forwarding-based denial-of-service attacks in wsns, *International Journal of Distributed Sensor Networks* 2013 (08 2013). doi:10.1155/2013/205920.
- [7] N. Mohammed, K. Elleithy, Comparative analysis of selective forwarding attacks over wireless sensor networks, *International Journal of Computer Applications* 111 (2015) 27–38. doi:10.5120/19607-1472.
- [8] T. H. Hai, E. Huh, Detecting selective forwarding attacks in wireless sensor networks using two-hops neighbor knowledge, in: *2008 Seventh IEEE International Symposium on Network Computing and Applications, 2008*, pp. 325–331. doi:10.1109/NCA.2008.13.
- [9] B. R. Chandavarkar, A. V. Gadagkar, A framework for residual energy model in unetstack simulator for underwater sensor networks, in: *2020 5th International Conference on Computing, Communication and Security (ICCCS), 2020*, pp. 1–8. doi:10.1109/ICCCS49678.2020.9277035.
- [10] B. R. Chandavarkar, A. V. Gadagkar, Mitigating localization and neighbour spoofing attacks in underwater sensor networks, in: *2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT), 2020*, pp. 1–5. doi:10.1109/ICCCNT49239.2020.9225431.
- [11] M. Chitre, R. Bhatnagar, W.-S. Soh, Unetstack: An agent-based software stack and simulator for underwater networks, in: *2014 Oceans - St. John's, 2014*, pp. 1–10. doi:10.1109/OCEANS.2014.7003044.